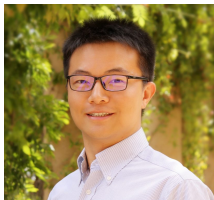


EfficientML.ai Lecture 05

Quantization

Part I



Song Han

Associate Professor, MIT
Distinguished Scientist, NVIDIA

 @SongHan_MIT



Lecture Plan

Today we will:

1. Review the numeric **data types**, including integers and floating-point numbers (FP32, FP16, INT4, etc.)
2. Learn the basic concept of **neural network quantization**
3. Learn three types of common neural network quantization:
 1. K-Means-based Quantization
 2. Linear Quantization
 3. Binary and Ternary Quantization

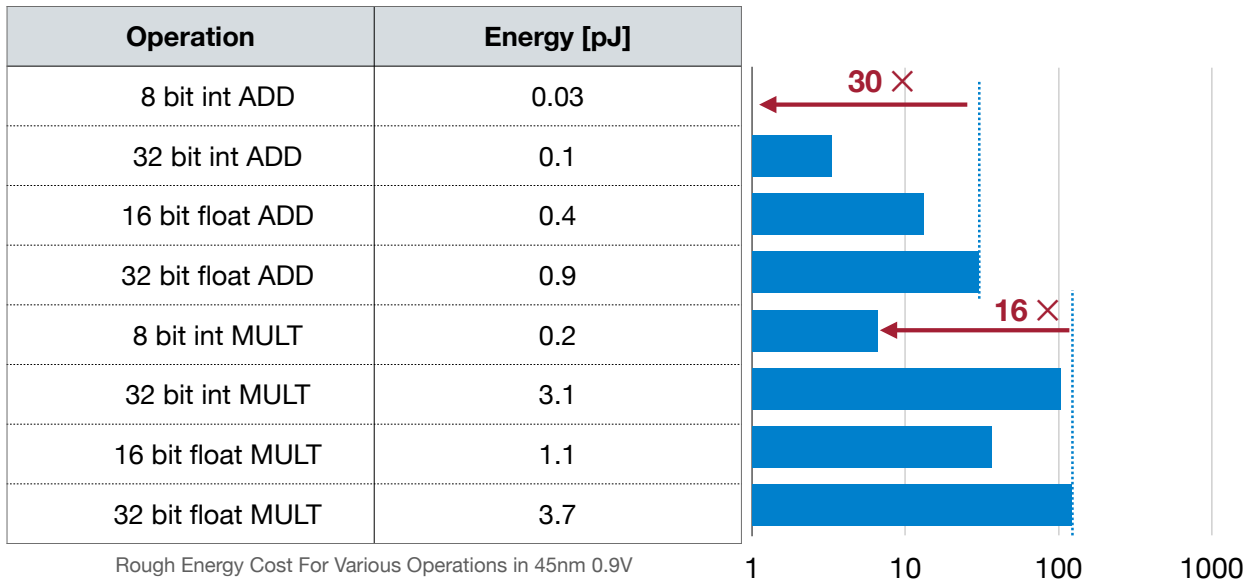
1	1	0	0	1	1	1	1
x	x	x	x	x	x	x	x

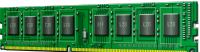
$$-2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = -49$$



Low Bit Operations are Cheaper

Less Bit-Width → Less Energy



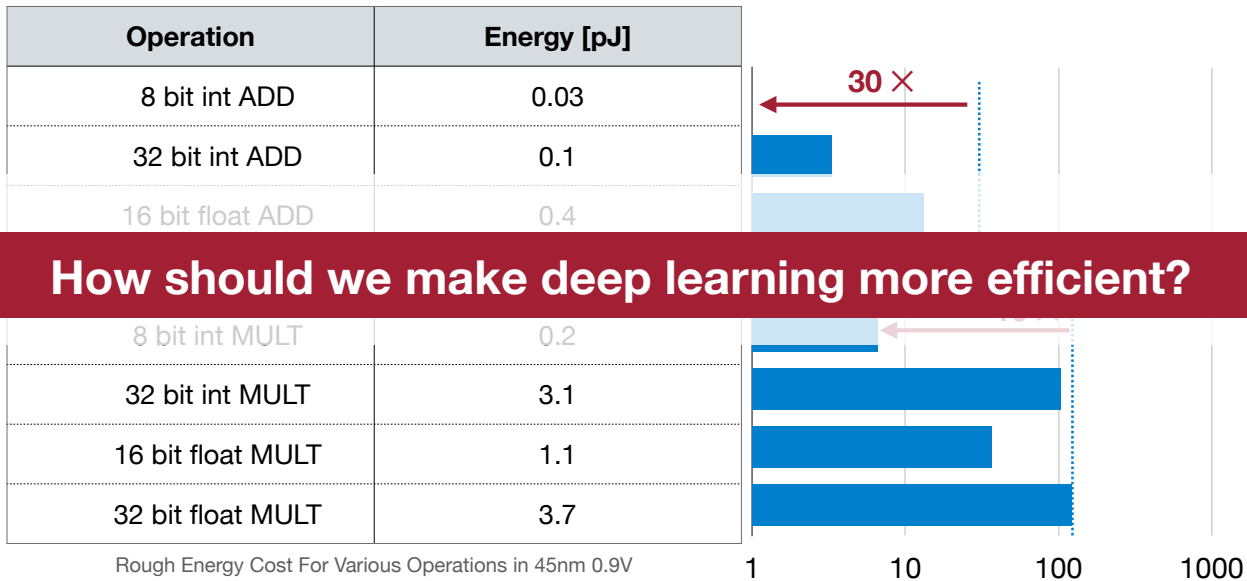
$$1 \text{  = 200 \times +$$

Computing's Energy Problem (and What We Can Do About it) [Horowitz, M., IEEE ISSCC 2014]

[This image](#) is in the public domain

Low Bit Operations are Cheaper

Less Bit-Width → Less Energy



How should we make deep learning more efficient?

Numeric Data Types

How is numeric data represented in modern computing systems?

Integer

- Unsigned Integer
 - n -bit Range: $[0, 2^n - 1]$
- Signed Integer
 - Sign-Magnitude Representation
 - n -bit Range: $[-2^{n-1} + 1, 2^{n-1} - 1]$
 - Both 000...00 and 100...00 represent 0
 - Two's Complement Representation
 - n -bit Range: $[-2^{n-1}, 2^{n-1} - 1]$
 - 000...00 represents 0
 - 100...00 represents -2^{n-1}

0	0	1	1	0	0	0	1
---	---	---	---	---	---	---	---

x x x x x x x x

$$2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = 49$$

Sign Bit

1	0	1	1	0	0	0	1
---	---	---	---	---	---	---	---

x x x x x x x x

$$-2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = -49$$

1	1	0	0	1	1	1	1
---	---	---	---	---	---	---	---

x x x x x x x x

$$-2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = -49$$

Fixed-Point Number



Integer . Fraction

"Decimal" Point



x x x x x x x x

$$-2^3 + 2^2 + 2^1 + 2^0 + 2^{-1} + 2^{-2} + 2^{-3} + 2^{-4} = 3.0625$$



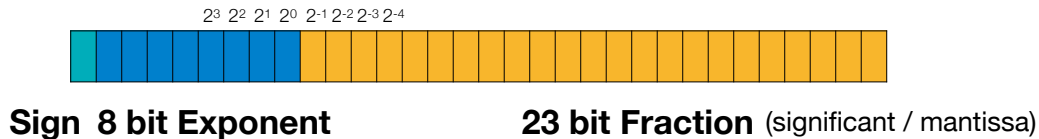
x x x x x x x x

$$(-2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0) \times 2^{-4} = 49 \times 0.0625 = 3.0625$$

(using 2's complement representation)

Floating-Point Number

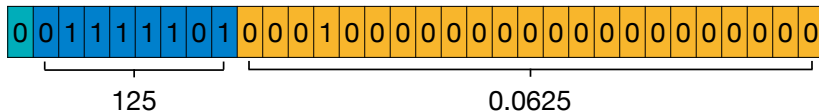
Example: 32-bit floating-point number in IEEE 754



$$(-1)^{\text{sign}} \times (1 + \text{Fraction}) \times 2^{\text{Exponent}-127} \quad \leftarrow \quad \text{Exponent Bias} = 127 = 2^{8-1}-1$$

How to represent **0.265625**?

$$0.265625 = 1.0625 \times 2^{-2} = (1 + 0.0625) \times 2^{125-127}$$



Floating-Point Number

Example: 32-bit floating-point number in IEEE 754



Sign 8 bit **Exponent**

23 bit Fraction (significant / mantissa)

$$(-1)^{\text{sign}} \times (1 + \text{Fraction}) \times 2^{\text{Exponent}-127} \quad \leftarrow \quad \text{Exponent Bias} = 127 = 2^{8-1}-1$$

How should we represent 0?

Floating-Point Number

Example: 32-bit floating-point number in IEEE 754



Sign 8 bit Exponent

23 bit Fraction

$$(-1)^{\text{sign}} \times (1 + \text{Fraction}) \times 2^{\text{Exponent}-127}$$

Normal Numbers, Exponent $\neq 0$

Should have been $(-1)^{\text{sign}} \times (1 + \text{Fraction}) \times 2^{0-127}$
But we force to be $(-1)^{\text{sign}} \times \text{Fraction} \times 2^{1-127}$

Subnormal Numbers, Exponent = 0



125

0.0625

$$0.265625 = 1.0625 \times 2^{-2} = (1 + 0.0625) \times 2^{125-127}$$



0

0



0

0

$$0 = 0 \times 2^{-126}$$

Floating-Point Number

Example: 32-bit floating-point number in IEEE 754



Sign 8 bit Exponent

23 bit Fraction

$$(-1)^{\text{sign}} \times (1 + \text{Fraction}) \times 2^{\text{Exponent}-127}$$

Normal Numbers, Exponent $\neq 0$

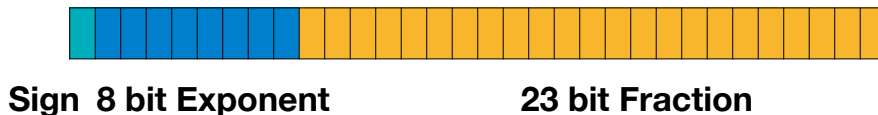
$$(-1)^{\text{sign}} \times \text{Fraction} \times 2^{1-127} \text{ 🧊}$$

Subnormal Numbers, Exponent=0

What is the smallest positive subnormal value?

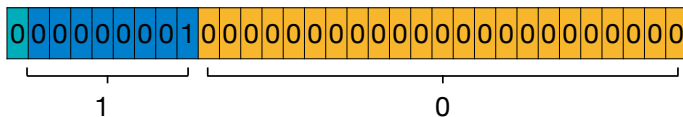
Floating-Point Number

Example: 32-bit floating-point number in IEEE 754



$$(-1)^{\text{sign}} \times (1 + \text{Fraction}) \times 2^{\text{Exponent}-127}$$

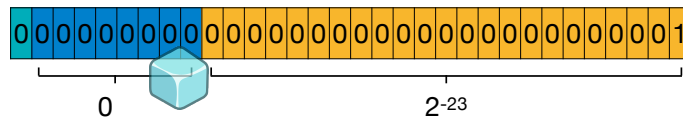
Normal Numbers, Exponent $\neq 0$



$$2^{-126} = (1 + 0) \times 2^{1-127}$$

$$(-1)^{\text{sign}} \times \text{Fraction} \times 2^{1-127}$$

Subnormal Numbers, Exponent=0



$$2^{-149} = 2^{-23} \times 2^{-126}$$

Floating-Point Number

Example: 32-bit floating-point number in IEEE 754



Sign 8 bit Exponent

23 bit Fraction

$$(-1)^{\text{sign}} \times (1 + \text{Fraction}) \times 2^{\text{Exponent}-127}$$

Normal Numbers, Exponent $\neq$ 0

$$(-1)^{\text{sign}} \times \text{Fraction} \times 2^{1-127} \text{ 🧊}$$

Subnormal Numbers, Exponent=0

What is the largest positive subnormal value?

Floating-Point Number

Example: 32-bit floating-point number in IEEE 754



Sign 8 bit Exponent

23 bit Fraction

$$(-1)^{\text{sign}} \times (1 + \text{Fraction}) \times 2^{\text{Exponent}-127}$$

Normal Numbers, Exponent $\neq 0$



$$2^{-126} = (1 + \underline{0}) \times 2^{1-127}$$

$$(-1)^{\text{sign}} \times \text{Fraction} \times 2^{1-127}$$

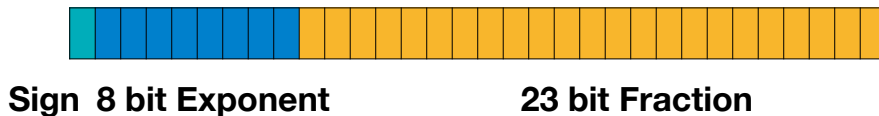
Subnormal Numbers, Exponent=0



$$2^{-126-2^{-149}} = (1 - 2^{-23}) \times 2^{-126}$$

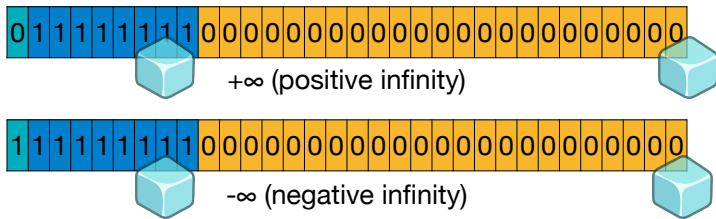
Floating-Point Number

Example: 32-bit floating-point number in IEEE 754



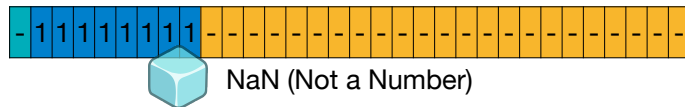
$$(-1)^{\text{sign}} \times (1 + \text{Fraction}) \times 2^{\text{Exponent}-127}$$

Normal Numbers, Exponent $\neq$ 0



$$(-1)^{\text{sign}} \times \text{Fraction} \times 2^{1-127}$$

Subnormal Numbers, Exponent=0



much waste. revisit in fp8.

Floating-Point Number

Example: 32-bit floating-point number in IEEE 754



Sign 8 bit Exponent

23 bit Fraction

Exponent	Fraction=0	Fraction≠0	Equation
00 _H = 0 	±0	subnormal	$(-1)^{\text{sign}} \times \text{Fraction} \times 2^{1-127}$
01 _H ... FE _H = 1 ... 254	normal		$(-1)^{\text{sign}} \times (1 + \text{Fraction}) \times 2^{\text{Exponent}-127}$
FF _H = 255 	±INF 	NaN	



Floating-Point Number

Exponent Width → Range; Fraction Width → Precision

IEEE 754 Single Precision 32-bit Float (IEEE FP32)



IEEE 754 Half Precision 16-bit Float (IEEE FP16)



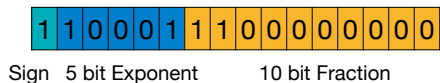
Google Brain Float (BF16)



Exponent (bits)	Fraction (bits)	Total (bits)
8	23	32
5	10	16
8	7	16

Numeric Data Types

- Question:** What is the following IEEE half precision (IEEE FP16) number in decimal?



Exponent Bias = 15_{10}

- Sign: -
- Exponent: $10001_2 - 15_{10} = 17_{10} - 15_{10} = 2_{10}$
- Fraction: $1100000000_2 = 0.75_{10}$
- Decimal Answer = $-(1 + 0.75) \times 2^2 = -1.75 \times 2^2 = -7.0_{10}$

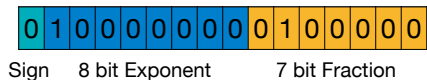
Numeric Data Types

- **Question:** What is the decimal 2.5 in Brain Float (BF16)?

$$2.5_{10} = 1.\underline{25}_{10} \times 2^1$$

Exponent Bias = 127_{10}

- Sign: +
- Exponent Binary: $1_{10} + 127_{10} = 128_{10} = 10000000_2$
- Fraction Binary: $0.25_{10} = 0100000_2$
- Binary Answer



Floating-Point Number

Exponent Width → Range; Fraction Width → Precision

IEEE 754 Single Precision 32-bit Float (IEEE FP32)



Exponent
(bits)

8

Fraction
(bits)

23

Total
(bits)

32

IEEE 754 Half Precision 16-bit Float (IEEE FP16)



5

10

16

Nvidia FP8 (E4M3)



- * FP8 E4M3 does not have INF, and $S.1111.111_2$ is used for NaN.
- * Largest FP8 E4M3 normal value is $S.1111.110_2 = 448$.

4

3

8

Nvidia FP8 (E5M2) for gradient in the backward



- * FP8 E5M2 have INF ($S.11111.00_2$) and NaN ($S.11111.XX_2$).
- * Largest FP8 E5M2 normal value is $S.11110.11_2 = 57344$.

5

2

8

INT4 and FP4

Exponent Width → Range; Fraction Width → Precision

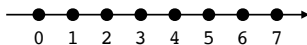
INT4

S			
0	0	0	1
0	1	1	1

=1

=7

-1, -2, -3, -4, -5, -6, -7, -8
0, 1, 2, 3, 4, 5, 6, 7



-1, -2, -3, -4, -5, -6, -7, -8
0, 1, 2, 3, 4, 5, 6, 7

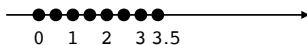
FP4 (E1M2)

S	E	M	M
0	0	0	1
0	1	1	1

=0.25×2¹⁻⁰=0.5

=(1+0.75)×2¹⁻⁰=3.5

-0, -0.5, -1, -1.5, -2, -2.5, -3, -3.5
0, 0.5, 1, 1.5, 2, 2.5, 3, 3.5



-0, -1, -2, -3, -4, -5, -6, -7 × 0.5
0, 1, 2, 3, 4, 5, 6, 7 × 0.5

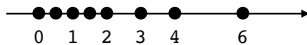
FP4 (E2M1)

S	E	E	M
0	0	0	1
0	1	1	1

=0.5×2¹⁻¹=0.5

=(1+0.5)×2³⁻¹=1

-0, -0.5, -1, -1.5, -2, -3, -4, -6
0, 0.5, 1, 1.5, 2, 3, 4, 6



-0, -1, -2, -3, -4, -6, -8, -12 × 0.5
0, 1, 2, 3, 4, 6, 8, 12 × 0.5

no inf, no NaN

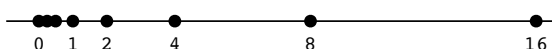
FP4 (E3M0)

S	E	E	E
0	0	0	1
0	1	1	1

=(1+0)×2¹⁻³=0.25

=(1+0)×2⁷⁻³=16

-0, -0.25, -0.5, -1, -2, -4, -8, -16
0, 0.25, 0.5, 1, 2, 4, 8, 16



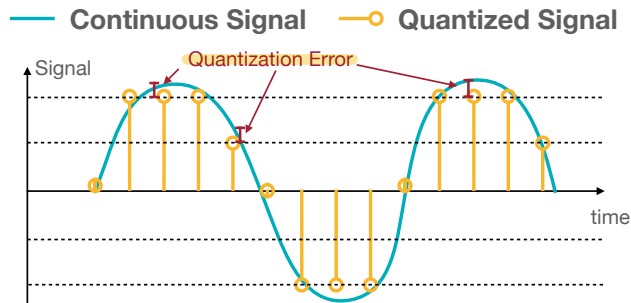
-0, -1, -2, -4, -8, -16, -32, -64 × 0.25
0, 1, 2, 4, 8, 16, 32, 64 × 0.25

no inf, no NaN

What is Quantization?

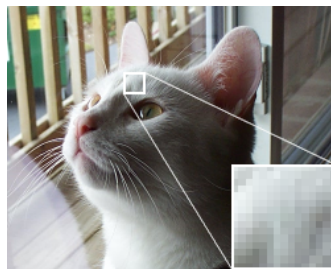
Quantization is the process of constraining an input from a continuous or otherwise large set of values to a discrete set.

目標在於 minimize Quantization Error

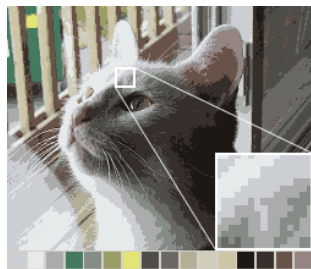


The difference between an input value and its quantized value is referred to as quantization error.

Original Image



16-Color Image



Images are in the public domain.
“Palettization”

Quantization [Wikipedia]

Neural Network Quantization: Agenda

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

3	0	2	1	3:	2.00
1	1	0	3	2:	1.50
0	3	1	0	1:	0.00
3	1	2	2	0:	-1.00

1	-2	0	-1
-1	-1	-2	1
-2	1	-1	-2
1	-1	0	0

$(\dots) - (-1) \times 1.07$

1	0	1	1
1	0	0	1
0	1	1	0
1	1	1	1

**K-Means-based
Quantization**

**Linear
Quantization**

**Binary/Ternary
Quantization**

Storage

Floating-Point
Weights

Computation

Floating-Point
Arithmetic

Neural Network Quantization: Agenda

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

3	0	2	1	3:	2.00
1	1	0	3	2:	1.50
0	3	1	0	1:	0.00
3	1	2	2	0:	-1.00

1	-2	0	-1
-1	-1	-2	1
-2	1	-1	-2
1	-1	0	0

$(\dots) - (-1) \times 1.07$

1	0	1	1
1	0	0	1
0	1	1	0
1	1	1	1

K-Means-based Quantization

Linear Quantization

Binary/Ternary Quantization

Storage	Floating-Point Weights	Integer Weights; Floating-Point Codebook
Computation	Floating-Point Arithmetic	Floating-Point Arithmetic

Neural Network Quantization

Weight Quantization

weights
(32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

Neural Network Quantization

Weight Quantization

weights
(32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

~~2.09, 2.12, 1.92, 1.87~~



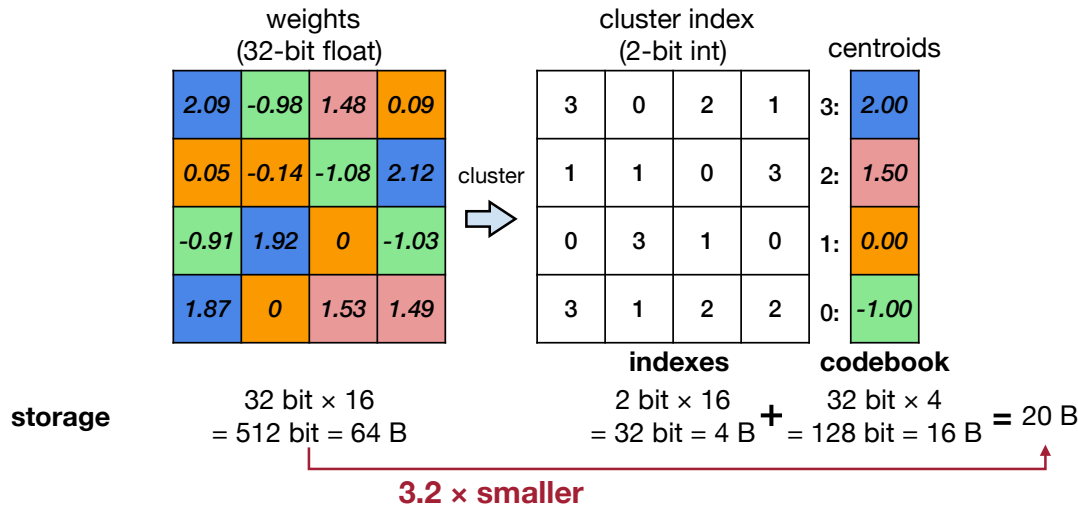
2.0

K-Means-based Weight Quantization

weights
(32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

K-Means-based Weight Quantization



Assume N -bit quantization, and #parameters = $M \gg 2^N$.

$$32 \text{ bit} \times M \\ = 32M \text{ bit}$$

$$N \text{ bit} \times M \\ = NM \text{ bit}$$

~~$$32 \text{ bit} \times 2^N \\ = 2^{N+5} \text{ bit}$$~~

32/N × smaller

Deep Compression [Han et al., ICLR 2016]

reconstructed weights (32-bit float)

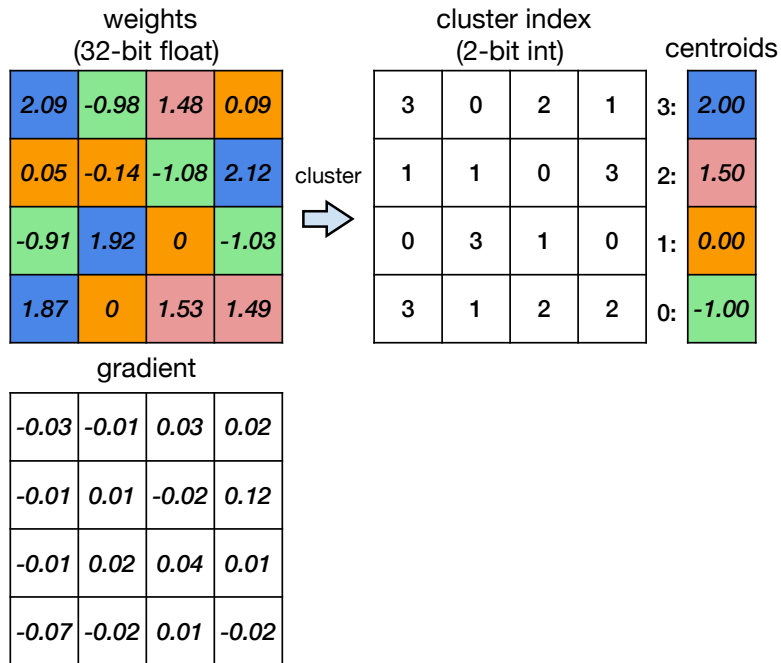
2.00	-1.00	1.50	0.00
0.00	0.00	-1.00	2.00
-1.00	2.00	0.00	-1.00
2.00	0.00	1.50	1.50

quantization error

0.09	0.02	-0.02	0.09
0.05	-0.14	-0.08	0.12
0.09	-0.08	0	-0.03
-0.13	0	0.03	-0.01

K-Means-based Weight Quantization

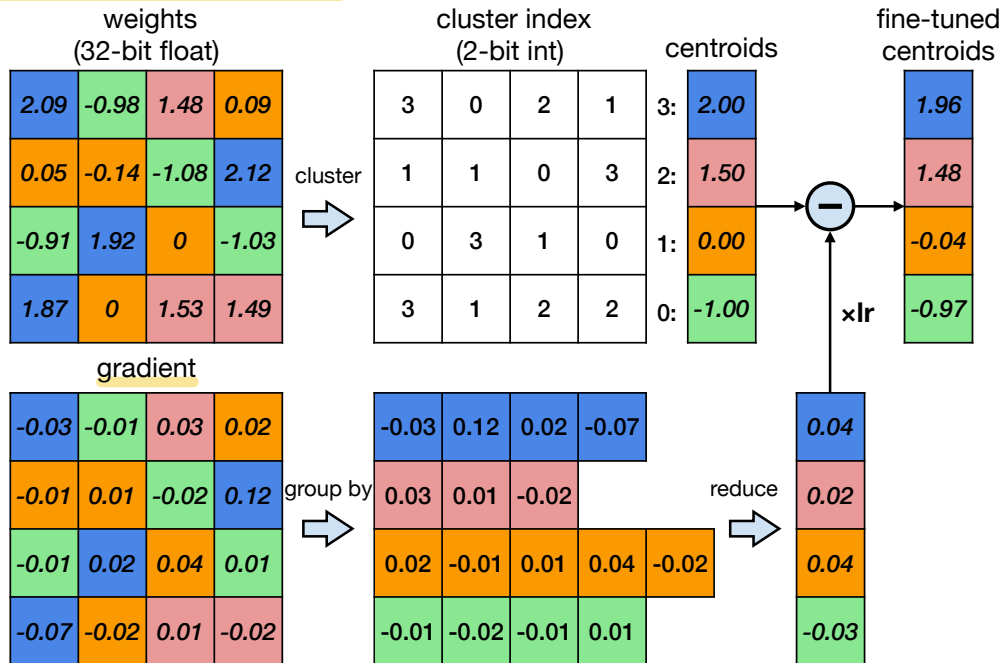
Fine-tuning Quantized Weights



Deep Compression [Han et al., ICLR 2016]

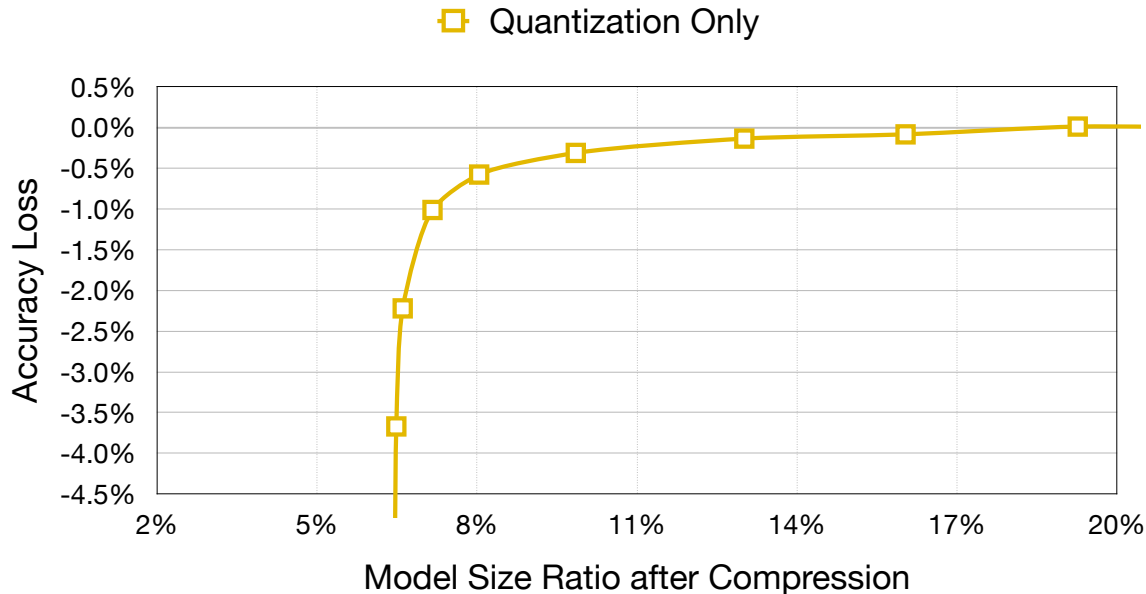
K-Means-based Weight Quantization

Fine-tuning Quantized Weights



K-Means-based Weight Quantization

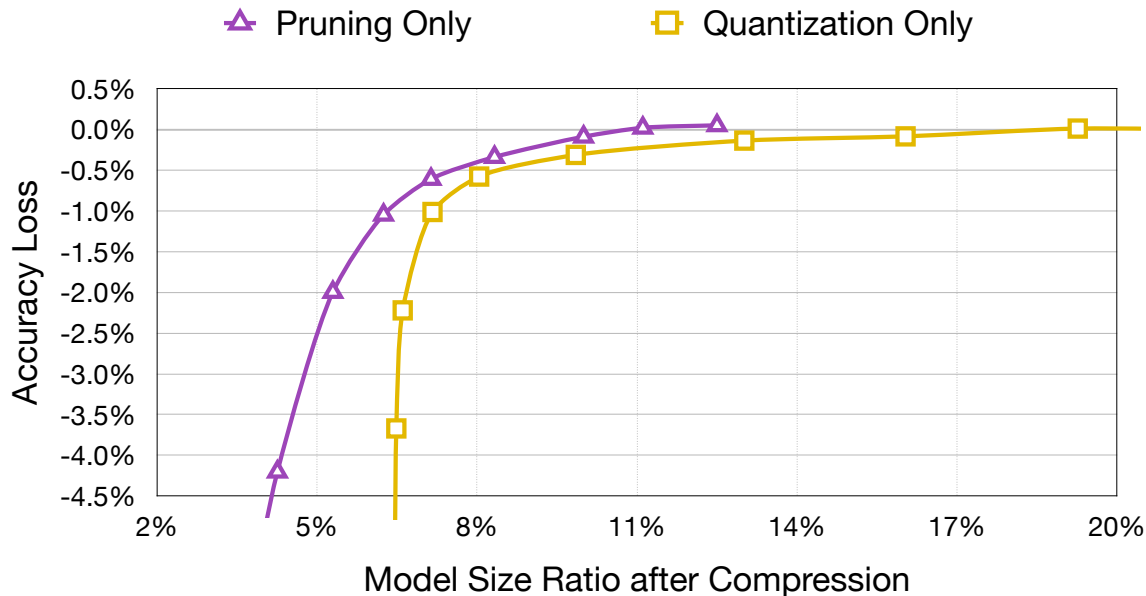
Accuracy vs. compression rate for AlexNet on ImageNet dataset



Deep Compression [Han et al., ICLR 2016]

K-Means-based Weight Quantization

Accuracy vs. compression rate for AlexNet on ImageNet dataset



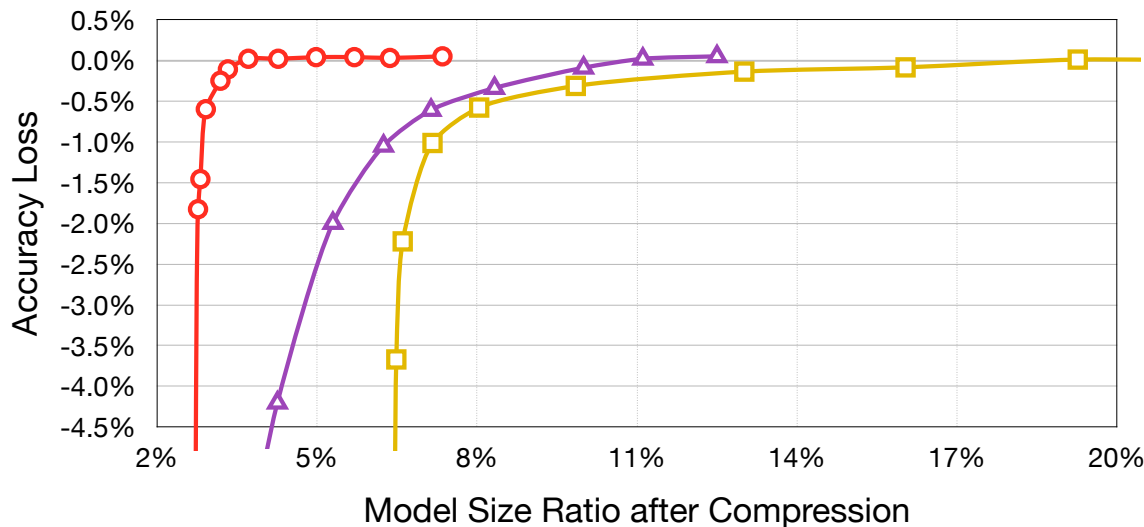
Deep Compression [Han et al., ICLR 2016]

K-Means-based Weight Quantization

Accuracy vs. compression rate for AlexNet on ImageNet dataset

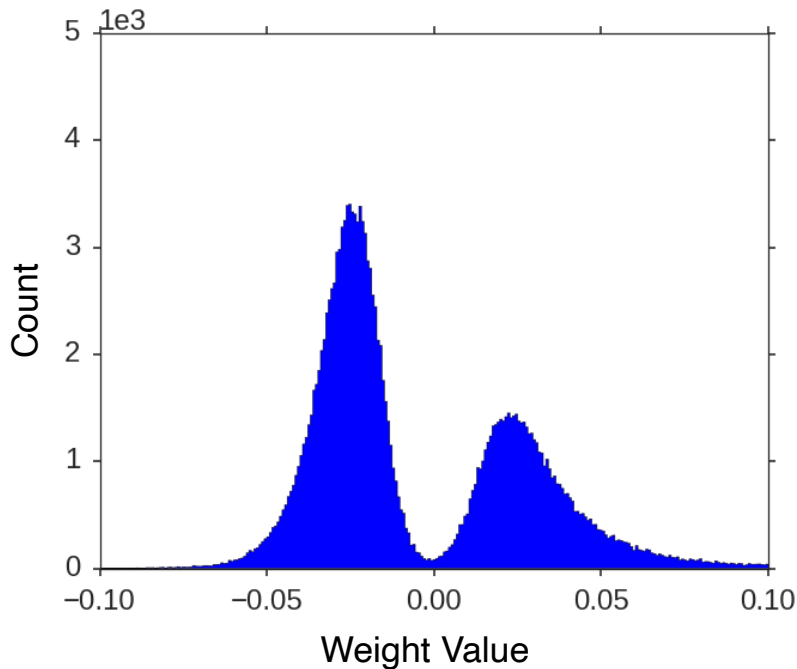
先做 pruning, 去除大量 redundant weights, 這樣需要 Quantize 的 weights 數減少了, 減少 Quantize 的工作量且減少 Quantize Error

○ Pruning + Quantization △ Pruning Only □ Quantization Only



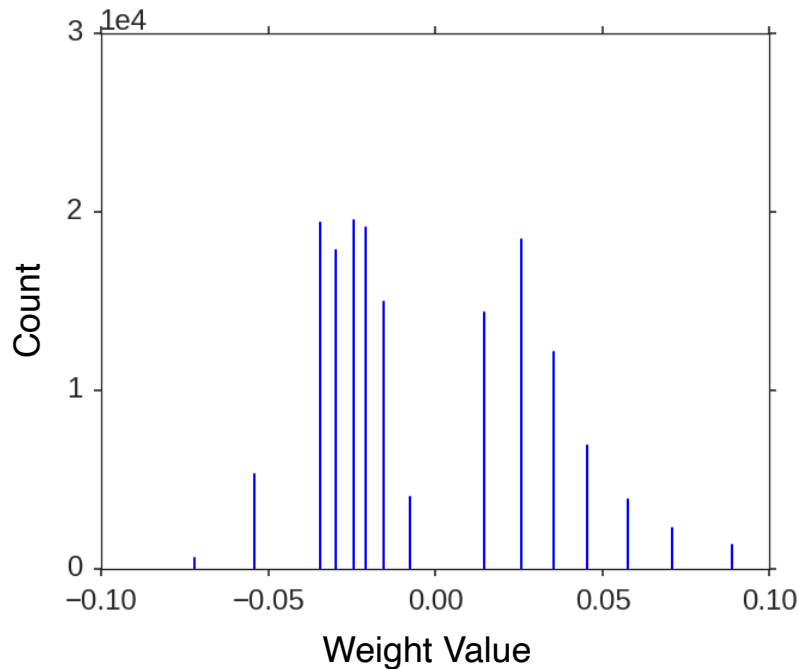
Deep Compression [Han et al., ICLR 2016]

Before Quantization: Continuous Weight



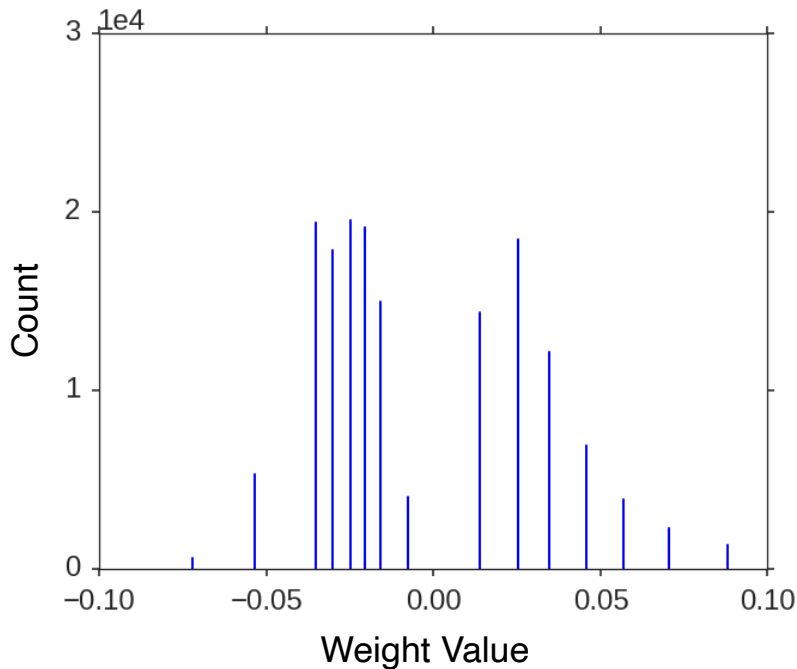
Deep Compression [Han *et al.*, ICLR 2016]

After Quantization: Discrete Weight



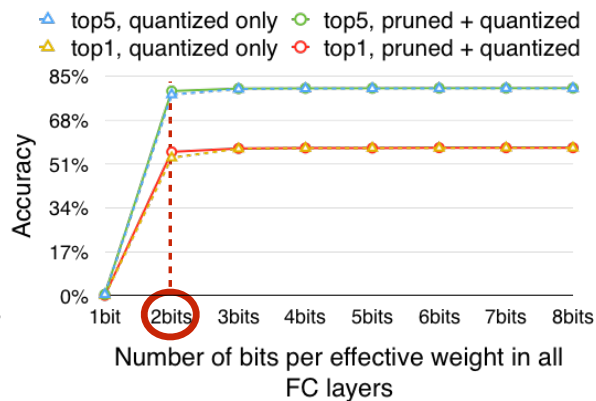
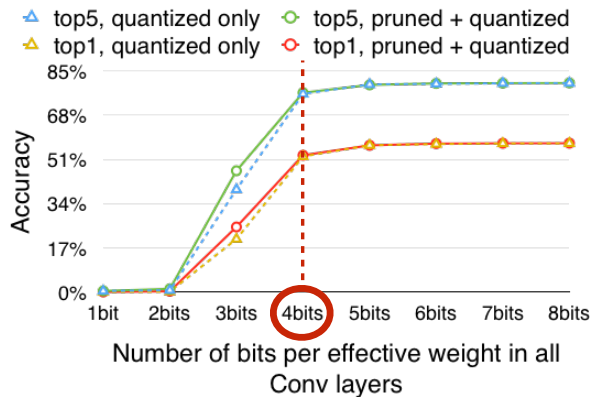
Deep Compression [Han *et al.*, ICLR 2016]

After Quantization: Discrete Weight after Retraining

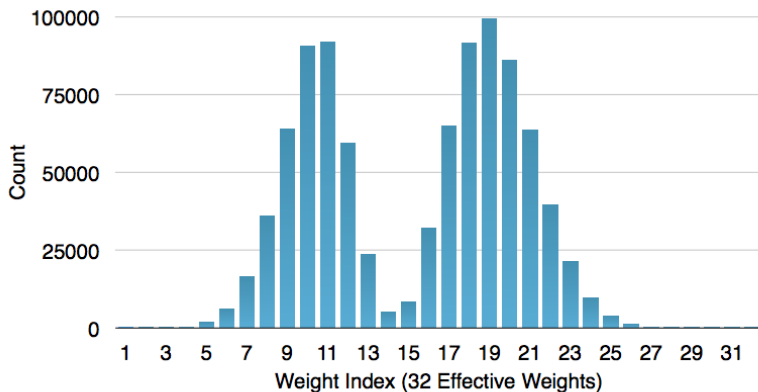
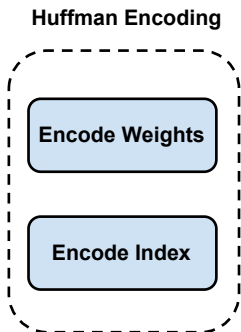


Deep Compression [Han *et al.*, ICLR 2016]

How Many Bits do We Need?

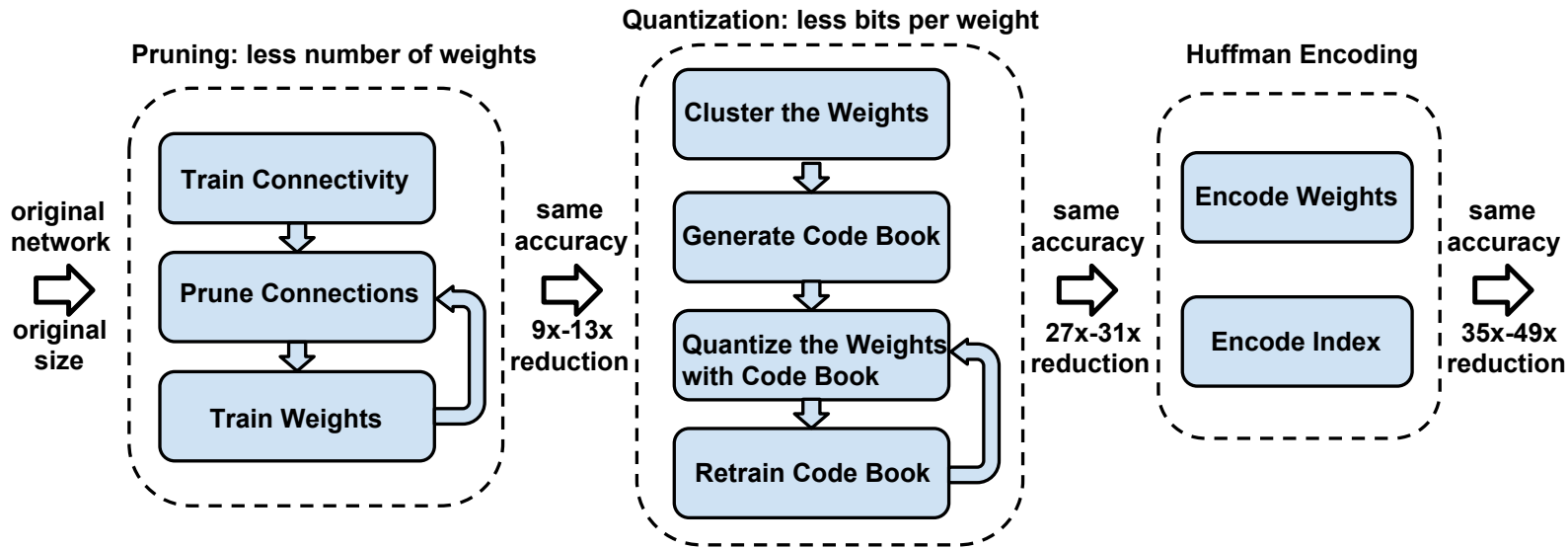


Huffman Coding



- In-frequent weights: use more bits to represent
- Frequent weights: use less bits to represent

Summary of Deep Compression



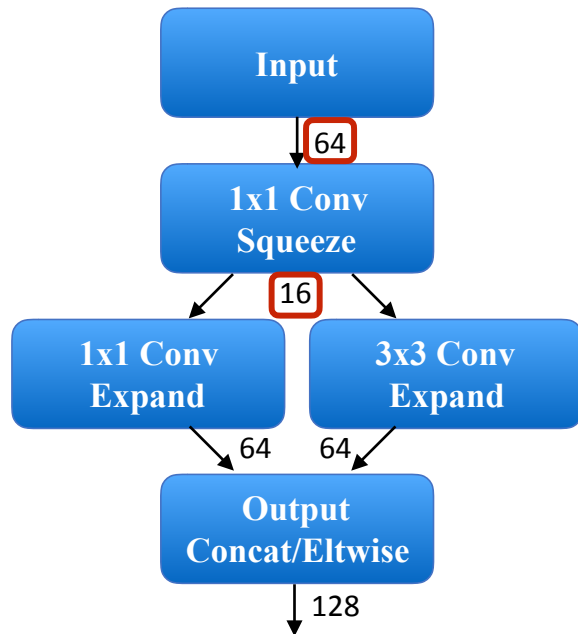
Deep Compression Results

Network	Original Size	Compressed Size	Compression Ratio	Original Accuracy	Compressed Accuracy
LeNet-300	1070KB	27KB	40x	98.36%	98.42%
LeNet-5	1720KB	44KB	39x	99.20%	99.26%
AlexNet	240MB	6.9MB	35x	80.27%	80.30%
VGGNet	550MB	11.3MB	49x	88.68%	89.09%
GoogleNet	28MB	2.8MB	10x	88.90%	88.92%
ResNet-18	44.6MB	4.0MB	11x	89.24%	89.28%

Can we make compact models to begin with?

Deep Compression [Han *et al.*, ICLR 2016]

SqueezeNet



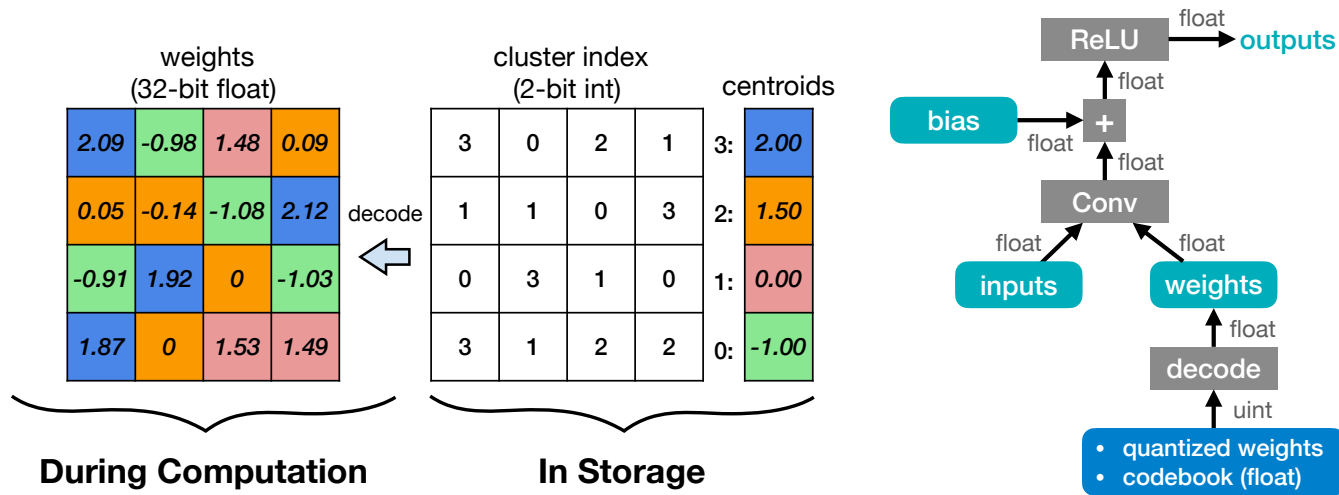
SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size [Iandola et al., arXiv 2016]

Deep Compression on SqueezeNet

Network	Approach	Size	Ratio	Top-1 Accuracy	Top-5 Accuracy
AlexNet	-	240MB	1x	<u>57.2%</u>	<u>80.3%</u>
AlexNet	SVD	48MB	5x	56.0%	79.4%
AlexNet	Deep Compression	6.9MB	35x	57.2%	80.3%
SqueezeNet	-	4.8MB	50x	57.5%	80.3%
SqueezeNet	Deep Compression	0.47MB	510x	<u>57.5%</u>	<u>80.3%</u>

SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size [Iandola et al., arXiv 2016]

K-Means-based Weight Quantization



- The weights are decompressed using a lookup table (*i.e.*, codebook) during runtime inference.
- K-Means-based Weight Quantization only saves storage cost of a neural network model.
 - All the computation and memory access are still floating-point.

Neural Network Quantization

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

3	0	2	1	3:	2.00
1	1	0	3	2:	1.50
0	3	1	0	1:	0.00
3	1	2	2	0:	-1.00

1	-2	0	-1
-1	-1	-2	1
-2	1	-1	-2
1	-1	0	0

$(\dots) - (-1) \times 1.07$

1	0	1	1
1	0	0	1
0	1	1	0
1	1	1	1

**K-Means-based
Quantization**

**Linear
Quantization**

**Binary/Ternary
Quantization**

Storage	Floating-Point Weights	Integer Weights; Floating-Point Codebook	Integer Weights
Computation	Floating-Point Arithmetic	Floating-Point Arithmetic	Integer Arithmetic

Linear Quantization

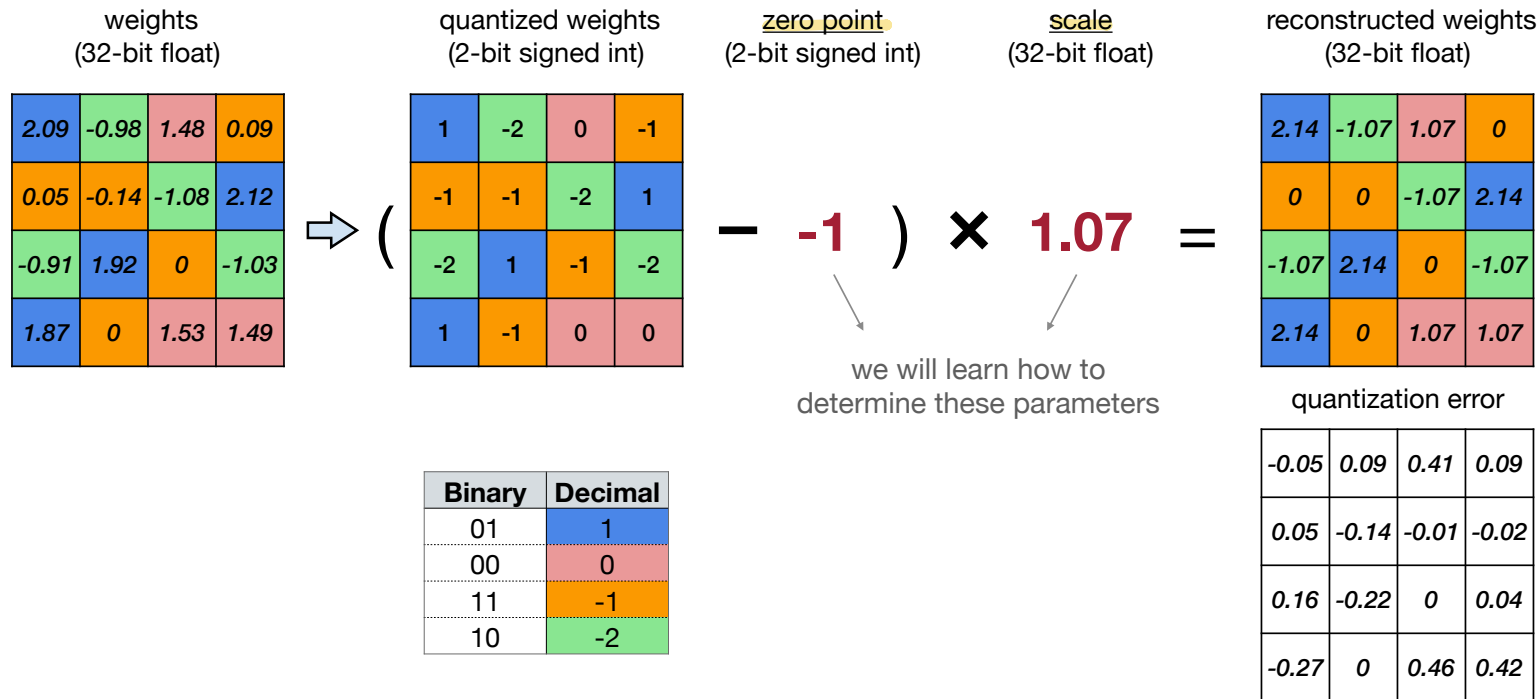
What is Linear Quantization?

weights
(32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

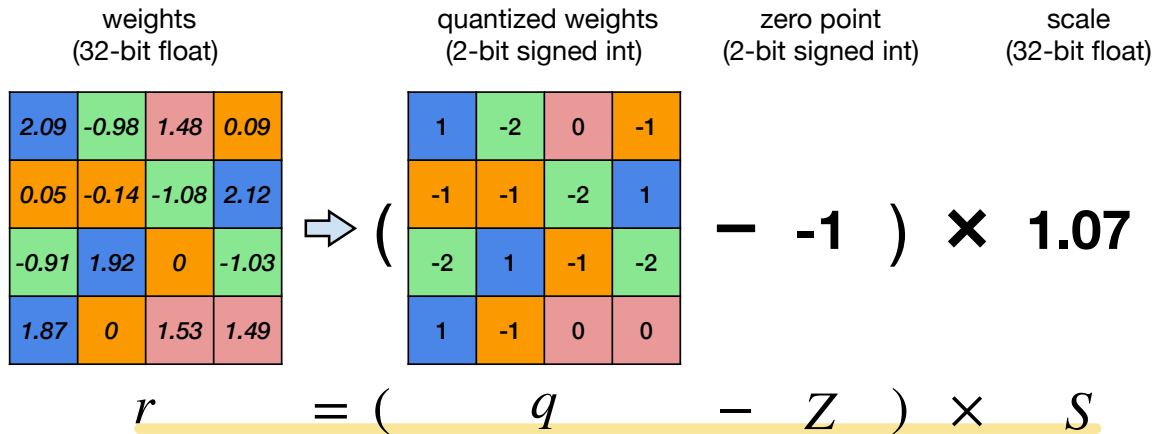
What is Linear Quantization?

An affine mapping of integers to real numbers



Linear Quantization

An affine mapping of integers to real numbers $r = S(q - Z)$



Floating-point

Integer

Integer

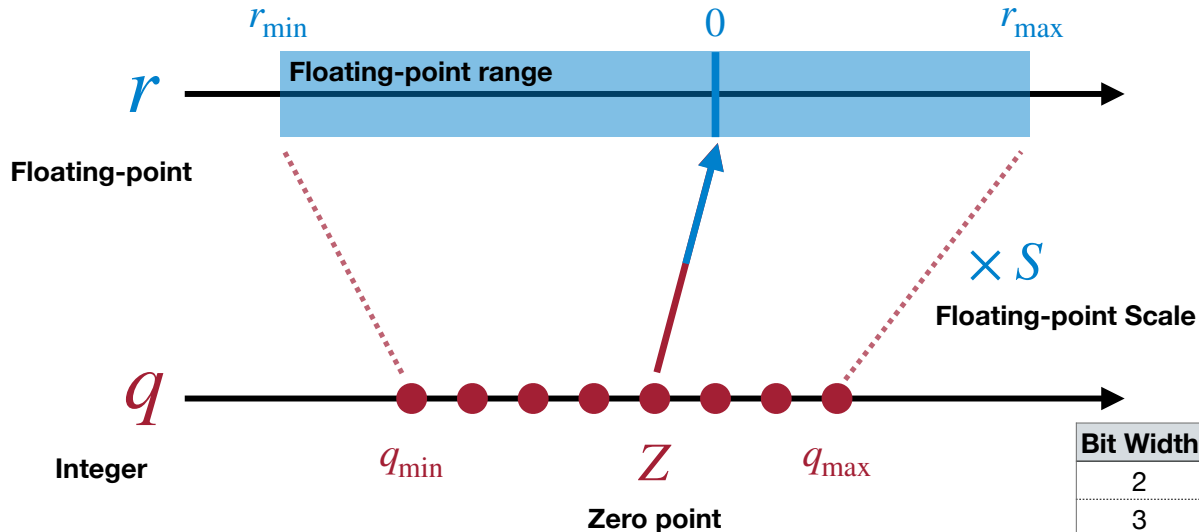
Floating-point

- quantization parameter
- allow real number $r=0$ be exactly representable by a quantized integer Z
- quantization parameter

Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob et al., CVPR 2018]

Linear Quantization

An affine mapping of integers to real numbers $r = S(q - Z)$

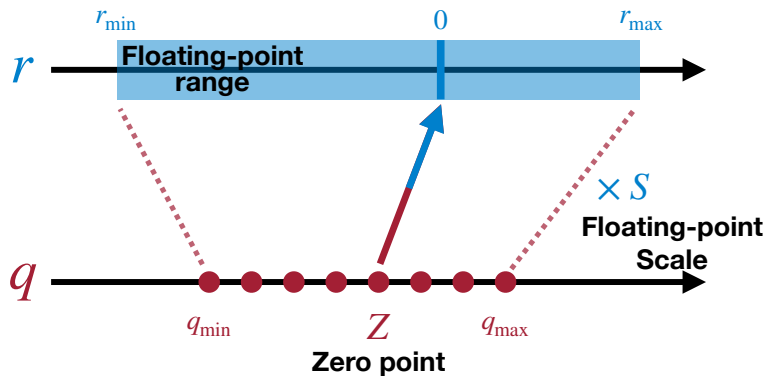


Bit Width	$q_{\min}$	$q_{\max}$
2	-2	1
3	-4	3
4	-8	7
N	-2^{N-1}	$2^{N-1}-1$

Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob et al., CVPR 2018]

Scale of Linear Quantization

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



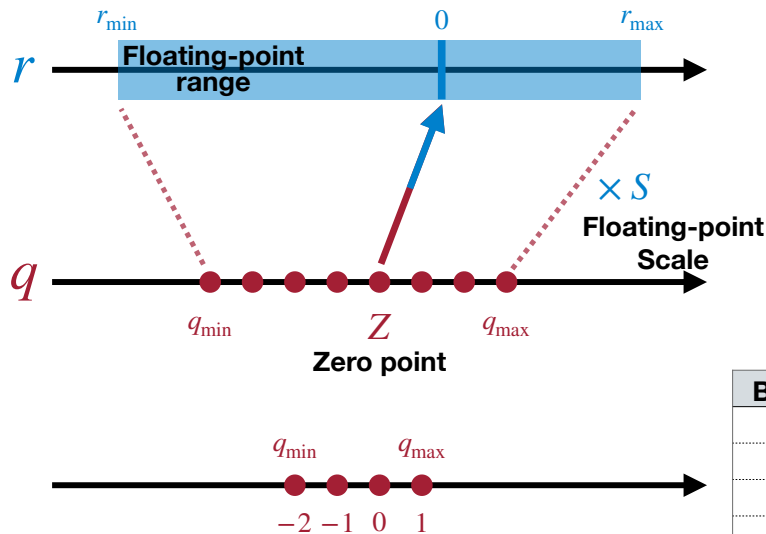
$$\begin{aligned} r_{\max} &= S(q_{\max} - Z) \\ r_{\min} &= S(q_{\min} - Z) \end{aligned}$$

$$\downarrow$$
$$r_{\max} - r_{\min} = S(q_{\max} - q_{\min})$$

$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}$$

Scale of Linear Quantization

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



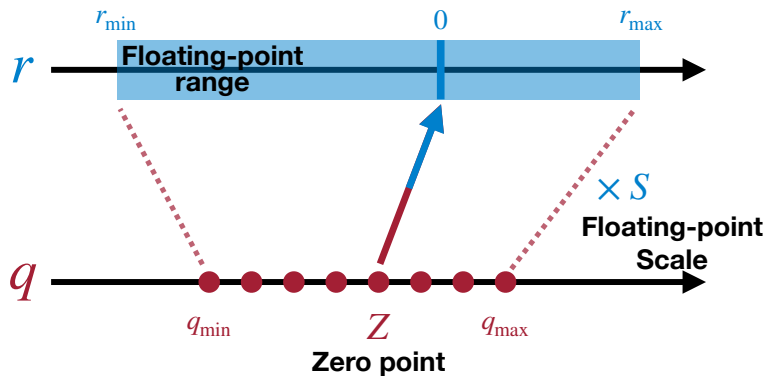
Binary	Decimal
01	1
00	0
11	-1
10	-2

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

$$\begin{aligned}
 S &= \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}} \\
 &= \frac{2.12 - (-1.08)}{1 - (-2)} \\
 &= 1.07
 \end{aligned}$$

Zero Point of Linear Quantization

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



$$\underline{r_{\min} = S(q_{\min} - Z)}$$

↓

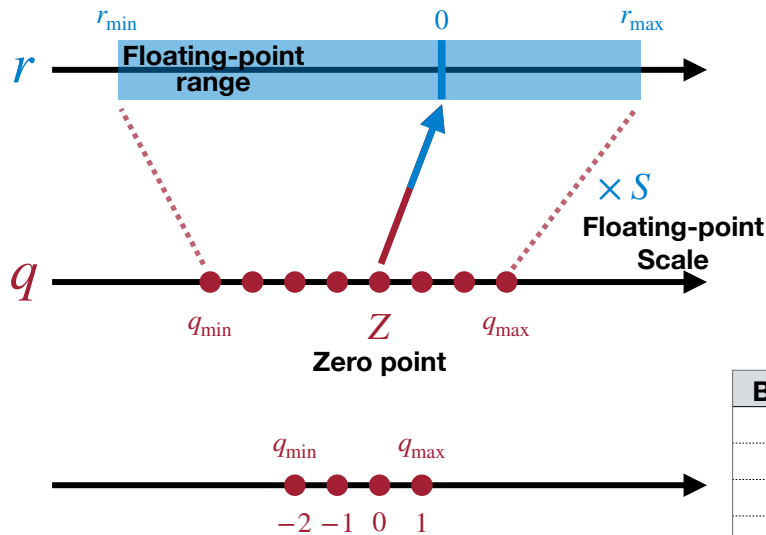
$$Z = q_{\min} - \frac{r_{\min}}{S}$$

↓

$$\underline{Z = \text{round} \left(q_{\min} - \frac{r_{\min}}{S} \right)}$$

Zero Point of Linear Quantization

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

$$Z = q_{\min} - \frac{r_{\min}}{S}$$

Binary	Decimal
01	1
00	0
11	-1
10	-2

$$= \text{round}\left(-2 - \frac{-1.08}{1.07}\right)$$

$$= -1$$

Linear Quantized Matrix Multiplication

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$

- Consider the following matrix multiplication.

$$\mathbf{Y} = \mathbf{W}\mathbf{X}$$

$$S_Y (\mathbf{q}_Y - Z_Y) = S_W (\mathbf{q}_W - Z_W) \cdot S_X (\mathbf{q}_X - Z_X)$$

$$\mathbf{q}_Y = \frac{S_W S_X}{S_Y} (\mathbf{q}_W - Z_W) (\mathbf{q}_X - Z_X) + Z_Y$$

$$\mathbf{q}_Y = \frac{S_W S_X}{S_Y} (\mathbf{q}_W \mathbf{q}_X - Z_W \mathbf{q}_X - Z_X \mathbf{q}_W + Z_W Z_X) + Z_Y$$

Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob *et al.*, CVPR 2018]

Linear Quantized Matrix Multiplication

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$

- Consider the following matrix multiplication.

$$\mathbf{Y} = \mathbf{W}\mathbf{X}$$

$$\mathbf{q}_Y = \frac{S_W S_X}{S_Y} \left(\mathbf{q}_W \mathbf{q}_X - Z_W \mathbf{q}_X - Z_X \mathbf{q}_W + Z_W Z_X \right) + Z_Y$$

Precompute

N -bit Integer Multiplication
32-bit Integer Addition/Subtraction

N -bit Integer Addition

Linear Quantized Matrix Multiplication

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$

- Consider the following matrix multiplication.

$$\mathbf{Y} = \mathbf{W}\mathbf{X}$$

$$\mathbf{q}_Y = \frac{S_W S_X}{S_Y} (\mathbf{q}_W \mathbf{q}_X - Z_W \mathbf{q}_X - Z_X \mathbf{q}_W + Z_W Z_X) + Z_Y$$

- Empirically, the scale $\frac{S_W S_X}{S_Y}$ is always in the interval (0, 1).

Fixed-point Multiplication

$$\frac{S_W S_X}{S_Y} = 2^{-n} M_0, \text{ where } M_0 \in [0.5, 1)$$

Bit Shift

Linear Quantized Matrix Multiplication

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$

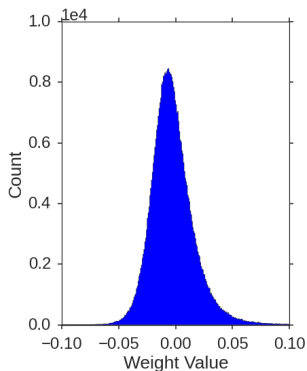
- Consider the following matrix multiplication.

$$\mathbf{Y} = \mathbf{W}\mathbf{X}$$

$$\mathbf{q}_Y = \frac{S_W S_X}{S_Y} \left(\mathbf{q}_W \mathbf{q}_X - Z_W \mathbf{q}_X - Z_X \mathbf{q}_W + Z_W Z_X \right) + Z_Y$$

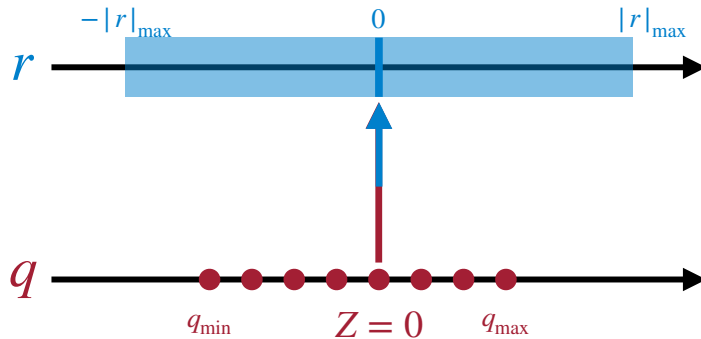
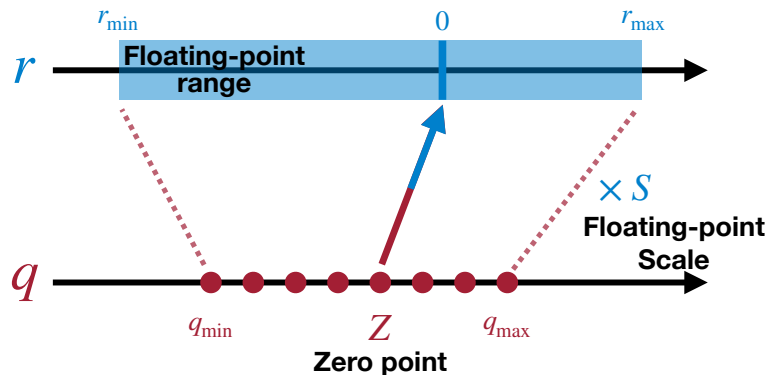
Rescale to N -bit Integer N -bit Integer Multiplication
32-bit Integer Addition/Subtraction N -bit Integer Addition

$$Z_W = 0?$$



Symmetric Linear Quantization

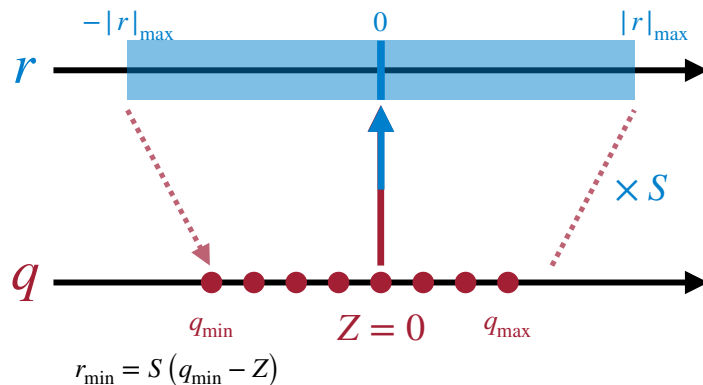
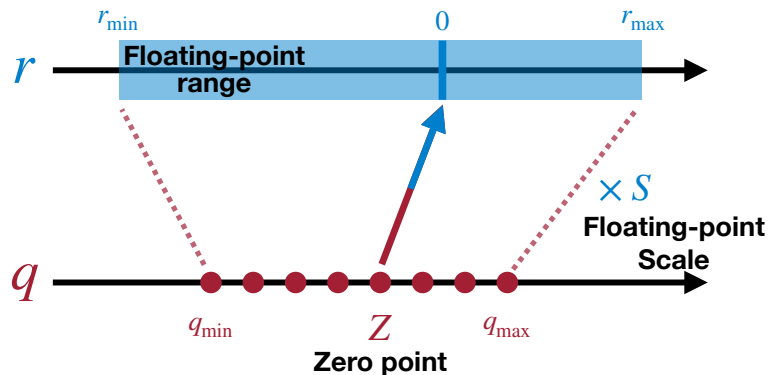
Zero point $Z = 0$ and Symmetric floating-point range



Bit Width	$q_{\min}$	$q_{\max}$
2	-2	1
3	-4	3
4	-8	7
N	-2^{N-1}	$2^{N-1}-1$

Symmetric Linear Quantization

Full range mode



$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}$$

$$S = \frac{r_{\min}}{q_{\min} - Z} = \frac{-|r|_{\max}}{q_{\min}} = \frac{|r|_{\max}}{2^{N-1}}$$

Bit Width	$q_{\min}$	$q_{\max}$
2	-2	1
3	-4	3
4	-8	7
N	-2^{N-1}	$2^{N-1}-1$

Linear Quantized Matrix Multiplication

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$

- Consider the following matrix multiplication, when $Z_W=0$.

$$Y = WX$$

The diagram illustrates the simplification of a quantized matrix multiplication formula. It starts with a general formula and then shows a simplified version with annotations.

General Formula:

$$q_Y = \frac{S_W S_X}{S_Y} (q_W q_X - Z_W q_X - Z_X q_W + Z_W Z_X) + Z_Y$$

Annotations for the General Formula:

- Rescale to N-bit Integer:** Points to the fraction $\frac{S_W S_X}{S_Y}$.
- N-bit Integer Multiplication:** Points to the term $q_W q_X$.
- 32-bit Integer Addition/Subtraction:** Points to the terms $-Z_W q_X$ and $-Z_X q_W$.
- Precompute:** Points to the term $Z_W Z_X$.
- N-bit Integer Addition:** Points to the term $+Z_Y$.

Simplified Formula (when $Z_W = 0$):

$$q_Y = \frac{S_W S_X}{S_Y} (q_W q_X - Z_X q_W) + Z_Y$$

Annotations for the Simplified Formula:

- $Z_W = 0$:** A blue arrow points from this text to the $-Z_W q_X$ term in the general formula, indicating its removal.
- Precompute:** A blue arrow points from this text to the $Z_W Z_X$ term in the general formula, indicating its removal.

Linear Quantized Fully-Connected Layer

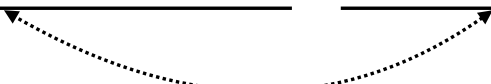
Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$

- So far, we ignore bias. Now we consider the following fully-connected layer with bias.

$$\mathbf{Y} = \mathbf{W}\mathbf{X} + \mathbf{b}$$

$$S_Y(\mathbf{q}_Y - Z_Y) = S_W(\mathbf{q}_W - Z_W) \cdot S_X(\mathbf{q}_X - Z_X) + S_b(\mathbf{q}_b - Z_b)$$

$$\downarrow Z_W = 0$$

$$S_Y(\mathbf{q}_Y - Z_Y) = \underbrace{S_W S_X(\mathbf{q}_W \mathbf{q}_X - Z_X \mathbf{q}_W)}_{\text{}} + \underbrace{S_b(\mathbf{q}_b - Z_b)}_{\text{}}$$


Linear Quantized Fully-Connected Layer

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$

- So far, we ignore bias. Now we consider the following fully-connected layer with bias.

$$\mathbf{Y} = \mathbf{W}\mathbf{X} + \mathbf{b}$$

$$S_Y(\mathbf{q}_Y - Z_Y) = S_W(\mathbf{q}_W - Z_W) \cdot S_X(\mathbf{q}_X - Z_X) + S_b(\mathbf{q}_b - Z_b)$$

$$\downarrow Z_W = 0$$

$$S_Y(\mathbf{q}_Y - Z_Y) = S_W S_X(\mathbf{q}_W \mathbf{q}_X - Z_X \mathbf{q}_W) + S_b(\mathbf{q}_b - Z_b)$$

$$\downarrow Z_b = 0, \quad S_b = S_W S_X$$

$$S_Y(\mathbf{q}_Y - Z_Y) = S_W S_X(\mathbf{q}_W \mathbf{q}_X - Z_X \mathbf{q}_W + \mathbf{q}_b)$$

Linear Quantized Fully-Connected Layer

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$

- So far, we ignore bias. Now we consider the following fully-connected layer with bias.

$$\mathbf{Y} = \mathbf{W}\mathbf{X} + \mathbf{b}$$

$$Z_{\mathbf{W}} = 0 \downarrow Z_{\mathbf{b}} = 0, \quad S_{\mathbf{b}} = S_{\mathbf{W}}S_{\mathbf{X}}$$

$$S_{\mathbf{Y}} (\mathbf{q}_{\mathbf{Y}} - Z_{\mathbf{Y}}) = S_{\mathbf{W}}S_{\mathbf{X}} (\mathbf{q}_{\mathbf{W}}\mathbf{q}_{\mathbf{X}} - Z_{\mathbf{X}}\mathbf{q}_{\mathbf{W}} + \mathbf{q}_{\mathbf{b}})$$

$$\mathbf{q}_{\mathbf{Y}} = \frac{S_{\mathbf{W}}S_{\mathbf{X}}}{S_{\mathbf{Y}}} (\mathbf{q}_{\mathbf{W}}\mathbf{q}_{\mathbf{X}} + \mathbf{q}_{\mathbf{b}} - Z_{\mathbf{X}}\mathbf{q}_{\mathbf{W}}) + Z_{\mathbf{Y}}$$

$$\downarrow \mathbf{q}_{bias} = \mathbf{q}_{\mathbf{b}} - Z_{\mathbf{X}}\mathbf{q}_{\mathbf{W}}$$

$$\mathbf{q}_{\mathbf{Y}} = \frac{S_{\mathbf{W}}S_{\mathbf{X}}}{S_{\mathbf{Y}}} (\mathbf{q}_{\mathbf{W}}\mathbf{q}_{\mathbf{X}} + \mathbf{q}_{bias}) + Z_{\mathbf{Y}}$$

We will discuss how to compute activation zero point in the next lecture.

Linear Quantized Fully-Connected Layer

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$

- So far, we ignore bias. Now we consider the following fully-connected layer with bias.

$$\mathbf{Y} = \mathbf{WX} + \mathbf{b}$$

$$Z_W = 0$$

$$Z_b = 0, \quad S_b = S_W S_X$$

$$\mathbf{q}_{bias} = \mathbf{q}_b - Z_X \mathbf{q}_W$$

$$\mathbf{q}_Y = \frac{S_W S_X}{S_Y} \left(\mathbf{q}_W \mathbf{q}_X + \mathbf{q}_{bias} \right) + Z_Y$$

Rescale to N -bit Int N -bit Int Mult. N -bit Int
32-bit Int Add. Add

Note: both $\mathbf{q}_b$ and $\mathbf{q}_{bias}$ are 32 bits.

Linear Quantized Convolution Layer

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$

- Consider the following convolution layer.

$$Y = \text{Conv}(W, X) + b$$

$$\begin{aligned} Z_W &= 0 \\ Z_b &= 0, \quad S_b = S_W S_X \\ q_{bias} &= q_b - \text{Conv}(q_W, Z_X) \end{aligned}$$

$$q_Y = \frac{S_W S_X}{S_Y} \left(\text{Conv}(q_W, q_X) + q_{bias} \right) + Z_Y$$

Rescale to N -bit Int N -bit Int Mult. 32-bit Int Add. N -bit Int Add

Note: both q_b and q_{bias} are 32 bits.

Linear Quantized Convolution Layer

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$

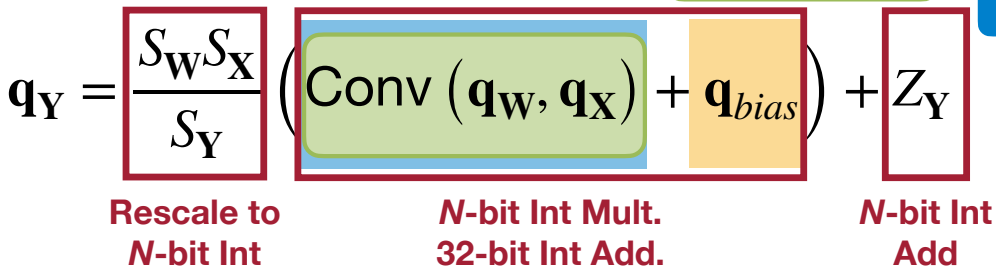
- Consider the following convolution layer.

$$Y = \text{Conv}(W, X) + b$$

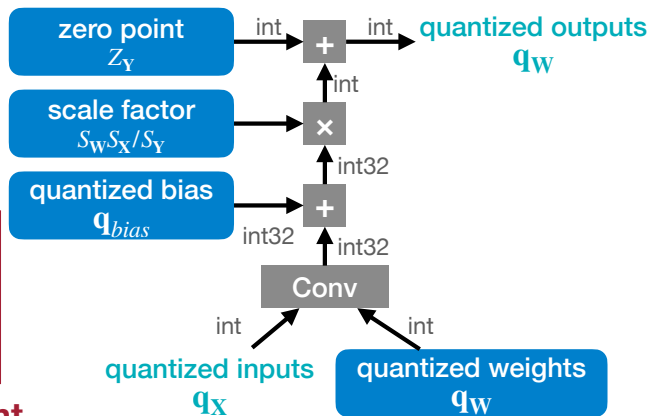
$$Z_W = 0$$

$$Z_b = 0, \quad S_b = S_W S_X$$

$$q_{bias} = q_b - \text{Conv}(q_W, Z_X)$$



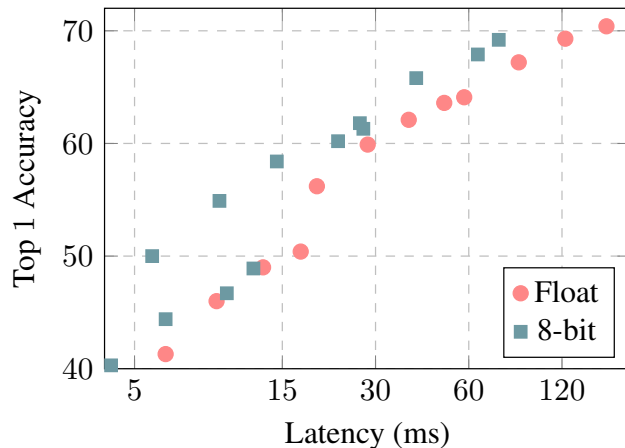
Note: both q_b and q_{bias} are 32 bits.



INT8 Linear Quantization

An affine mapping of integers to real numbers $r = S(q - Z)$

Neural Network	ResNet-50	Inception-V3
Floating-point Accuracy	76.4%	78.4%
8-bit Integer-quantized Accuracy	74.9%	75.4%



Latency-vs-accuracy tradeoff of float vs. integer-only MobileNets on ImageNet using Snapdragon 835 big cores.

Neural Network Quantization

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

3	0	2	1	3:	2.00
1	1	0	3	2:	1.50
0	3	1	0	1:	0.00
3	1	2	2	0:	-1.00

1	-2	0	-1
-1	-1	-2	1
-2	1	-1	-2
1	-1	0	0

$(\dots) - (-1) \times 1.07$

**K-Means-based
Quantization**

**Linear
Quantization**

Storage	Floating-Point Weights	Integer Weights; Floating-Point Codebook	Integer Weights
Computation	Floating-Point Arithmetic	Floating-Point Arithmetic	Integer Arithmetic



Summary of Today's Lecture

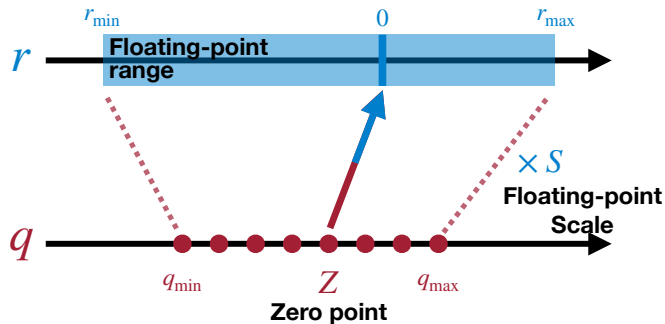
Today, we reviewed and learned

- the numeric data types used in the modern computing systems, including integers and floating-point numbers.
- the basic concept of **neural network quantization**: *converting the weights and activations of neural networks into a limited discrete set of numbers.*
- two types of common neural network quantization:
 - K-Means-based Quantization
 - Linear Quantization

1	1	0	0	1	1	1	1
---	---	---	---	---	---	---	---

$\times \quad \times \quad \times \quad \times \quad \times \quad \times \quad \times \quad \times$

$$-2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = -49$$



References

1. Model Compression and Hardware Acceleration for Neural Networks: A Comprehensive Survey [Deng et al., IEEE 2020]
2. Computing's Energy Problem (and What We Can Do About it) [Horowitz, M., IEEE ISSCC 2014]
3. Deep Compression [Han et al., ICLR 2016]
4. Neural Network Distiller: https://intellabs.github.io/distiller/algorithm_quantization.html
5. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob et al., CVPR 2018]
6. BinaryConnect: Training Deep Neural Networks with Binary Weights during Propagations [Courbariaux et al., NeurIPS 2015]
7. Binarized Neural Networks: Training Deep Neural Networks with Weights and Activations Constrained to +1 or -1. [Courbariaux et al., Arxiv 2016]
8. XNOR-Net: ImageNet Classification using Binary Convolutional Neural Networks [Rastegari et al., ECCV 2016]
9. Ternary Weight Networks [Li et al., Arxiv 2016]
10. Trained Ternary Quantization [Zhu et al., ICLR 2017]